面试技术拷打手册 · INTERVIEW HANDBOOK

Prism

以刁钻面试官视角对本项目的逐层拷打：架构选型、网关与限流、Redis、MySQL、MQ、WebFlux、RAG 与 Agent 共 38 题，附追问链与参考回答，以及 7 个已知弱点的应答话术与 4 个实战排障案例。

作者：夏梦楠 · 2026 年 10 月 · 配套仓库

github.com/purely628

目录

0. 电梯稿（先背熟） 3

A. 整体架构与选型 3

1. A1. 为什么拆 5 个服务？这是不是过度设计？ 3
2. A2. 为什么 MySQL 三个服务共库？【高危】 3
3. A3. 为什么网关用 Spring Cloud Gateway 而不是 Nginx 做所有事？ 3
4. A4. local 模式用 Redis Stream 替代 RabbitMQ，是不是玩具？【高危】 4

B. 网关层 4

5. B1. 令牌桶 vs 漏桶 vs 计数器？ 4
6. B2. Lua 脚本原子性的原理？会阻塞 Redis 吗？ 4
7. B3. JWT 放网关校验，下游信任 X-User-Id 头，安全吗？【高危】 4
8. B4. JWT 怎么做登出/改密失效？ 4
9. B5. SSE 经过网关有什么坑？ 4

C. Redis 深入 5

10. C1. 缓存穿透/击穿/雪崩在本项目哪里体现？ 5
11. C2. 语义缓存怎么设计的？相似度阈值怎么定？ 5
12. C3. 配额扣减为什么不用分布式锁？ 5
13. C4. Redis 挂了系统什么表现？ 5

D. MySQL 5

14. D1. t_usage_record 的索引怎么设计的？ 5
15. D2. 攒批写入的细节？为什么 500 条或 2 秒？ 6
16. D3. 对账怎么做的？发现差异怎么修？ 6

E. MQ 与异步 6

17. E1. 计量为什么走 MQ 而不是同步写库？ 6
18. E2. 消息丢失的每个环节？ 6
19. E3. 消息重复怎么办？ 6
20. E4. 为什么 usage 不怕乱序、kb 任务怕不怕？ 6

F. WebFlux 与并发 7

21. F1. chat 服务为什么用 WebFlux 而不是 MVC？ 7
22. F2. SSE 透传时怎么做 usage 统计？ 7
23. F3. 背压在 SSE 场景有意义吗？ 7

G. 业务与计费 7
24. G1. 配额为什么是软限制？硬限制怎么做？ 7
25. G2. Token 怎么计量的？误差多大？ 7
26. G3. 账单金额怎么算的？一致性怎么保证？ 7
H. RAG 与 Agent（Python 侧） 8
27. H1. 文档切分策略？chunk 大小怎么影响效果？ 8
28. H2. 召回之后为什么还要过滤？怎么引用溯源？ 8
29. H3. embedding 为什么默认哈希实现？生产换什么？ 8
30. H4. ReAct 循环讲一下？和 Function Calling 路线比？ 8
31. H5. Java 和 Python 的分工边界？跨语言任务怎么做可靠性？ 8
I. 工程与运维 8
32. I1. 双 profile（local/compose）的设计意图？ 8
33. I2. Nginx 配置里 SSE 相关的三件套？ 9
34. I3. 这个项目上线前还缺什么？（必考） 9
J. 已知弱点与应答话术（【高危】必背） 9
L. 实战踩坑录（E2E 真实排障，面试最出彩的部分） 9
35. L1. Redis 消费线程每 3 秒报 "command timed out" 9
36. L2. @RequestParam 全部 500：'-parameters' 10
37. L3. Java→Python 的 POST 请求体"消失"（422 body missing） 10
38. L4. 配额计数器漂移 500 = 250×2（活生生的对账案例） 10
K. 简历上怎么写这个项目 10
M. 本手册使用方法 11

以"刁钻面试官"视角对本项目的逐层拷打：问题 -> 追问链 -> 参考回答 -> 加/减分信号。

用法：每道题先自己作答，再对照参考回答；带 【高危】 的题是本项目已知弱点，必须提前准备"诚实+演进方案"话术。

0. 电梯稿（先背熟）

30 秒版：我做了一个 LLM 能力开放平台——对外提供 OpenAI 兼容的 chat 接口，企业客户端只需要改 base_url 和 API Key 就能接入；平台内部用 Spring Cloud Gateway 做鉴权和 Redis+Lua 令牌桶限流，chat 服务做多渠道路由、故障转移和语义缓存，用量走消息队列异步记账，按月配额管控、每日对账；同时平台内置了基于 LangChain 的 RAG 知识库和 ReAct Agent 能力，由 Python 服务承担 AI 编排。Java 做工程底座，Python 做模型编排。

2 分钟版：在 30 秒版基础上展开三点：①为什么做（企业接大模型的真痛点是接入后的治理：密钥、限流、成本、多模型切换，而不是调通 API）；②最难的部分（WebFlux 全链路 SSE 透传的同时做 usage 统计与语义缓存判断；以及计量链路的最终一致性设计：异步消息 + 双重幂等 + 攒批写入 + 每日对账）；③与烂大街项目的差异（秒杀是流量侧老三样，这个项目是"AI 中台"侧的网关+计费+RAG，且 Java/Python 分工是企业里真实的架构分工）。

A. 整体架构与选型

1. A1. 为什么拆 5 个服务？这是不是过度设计？

- 追问链：单体能做吗 -> 服务边界怎么定的 -> 拆分的收益/成本
- 参考回答：按变更频率与资源特征拆：user(低频管理)、chat(IO 密集流式转发，响应式栈)、billing(削峰写库，独立伸缩)、kb(文件+任务编排)、agent(Python 技术栈天然独立)。chat 是流量大头而 billing 是写入大头，两者伸缩模型完全不同，合在一起会互相拖累。诚实说：演示规模下单体+模块化也够，微服务的收益要在流量和团队规模上来才兑现——但接入网关+异步计费这个架构本身在单体里也成立，说明我理解的是架构模式而不是"为微服务而微服务"。
- 加分信号：能说出"拆的是架构边界而不只是进程边界"；减分信号：说不出每个服务独立存在的理由。

2. A2. 为什么 MySQL 三个服务共库？【高危】

- 参考回答：演示环境妥协，生产应按服务拆库或走服务间接口（billing 要 appName 就不该直读 t_app）。我把它列为已知债务并在代码注释标注。进一步能讲：拆库后 topApps 数据可以用冗余字段（usage 事件里带 appName 快照）或服务间查询+缓存解决。
- 减分信号：没意识到这是问题。

3. A3. 为什么网关用 Spring Cloud Gateway 而不是 Nginx 做所有事？

- 参考回答：Nginx 在最外层做静态资源、反代、连接层限流（limit_req 按IP）；业务层限流（按应用 Key、从 Redis 读配额）、鉴权、路由到注册中心的服务——这些是应用感知的逻辑，放 SCG 用 Java 写过滤器最顺。两层限流各司其职：Nginx 防 DDoS 级流量，SCG 做租户级配额。

4. A4. local 模式用 Redis Stream 替代 RabbitMQ，是不是玩具？【高危】

• 参考回答：接口抽象（UsageEventPublisher/UsageConsumer），实现可切换——这是依赖倒置，本地零中间件跑通全链路是为了开发体验和 CI；compose 模式上真实 RabbitMQ（有 DLQ、管理界面、集群）。Redis Stream 本身也有消费组/ACK/pending 语义，轻量场景是合理选型；差别在堆积能力、持久化级别、重试队列的完善度。能讲清"什么规模该换什么"是加分的。

B. 网关层

5. B1. 令牌桶 vs 漏桶 vs 计数器?

• 参考回答：计数器有临界突刺（窗口边界 2 倍流量）；漏桶恒速不适合突发对话流；令牌桶允许受控突发（capacity=2×rate），平均速率受控。Lua 里"按流逝时间补令牌+扣一个"必须原子，否则并发下超发。

• 追问：集群下为什么不能用 JVM 内存令牌桶？（多实例各自计数，总流量超限）-> Redis 集中执行 Lua，单分片原子。

6. B2. Lua 脚本原子性的原理？会阻塞 Redis 吗?

• 参考回答：Redis 单线程执行命令，Lua 脚本执行期间不插入其他命令，因此读-改-写天然原子；脚本是纯内存 O(1)（hash 两个 field），执行微秒级，不会造成可感知阻塞。减分回答：不知道 Lua 里不能写慢操作（如 KEYS 全扫）。

7. B3. JWT 放网关校验，下游信任 X-User-Id 头，安全吗？【高危】

• 参考回答：隐患是内网伪造——绕过网关直连服务就能伪造身份。演示级用"网络隔离+约定"，生产方案：①网关与下游 mTLS；②网关对转发头做签名（HMAC + 时间戳），下游验签；③或走 service mesh 的身份传播。能主动说出攻击路径再给方案是加分项。

8. B4. JWT 怎么做登出/改密失效?

• 参考回答：JWT 无状态、天然难失效。方案：短有效期+refresh token；黑名单（Redis 存 jti，网关 O(1) 查）；改密场景踢全部会话（按 uid 版本号）。项目里用了 12h 固定期，是演示简化。

9. B5. SSE 经过网关有什么坑?

• 参考回答：①不能有任何聚合 response body 的过滤器（ModifyResponseBody），否则把流变成整包；②超时配置（Netty/代理链每一层）；③Nginx 层要关 proxy_buffering，否则响应被缓冲成"一次性吐出"；④断连感知——客户端断开后取消上游订阅（WebFlux doOnCancel），否则白烧上游 token。这是本项目最硬的坑之一，能讲细节大加分。

C. Redis 深入

10. C1. 缓存穿透/击穿/雪崩在本项目哪里体现?

- 参考回答：app key 信息：穿透——恶意不存在的 key 打穿到 user 服务 -> 回源后写"不存在"标记（或布隆过滤器）；击穿——热点 app 的 hash 过期瞬间大量请求回源 -> 本项目 hash 不过期+变更时主动更新，规避了击穿；雪崩——大量 key 同时过期 -> TTL 加随机抖动。语义缓存过期加随机数同理。

11. C2. 语义缓存怎么设计的？相似度阈值怎么定?

- 追问链：为什么不用精确 hash -> 向量怎么存/怎么比 -> 误命中怎么办 -> 多实例怎么办
- 参考回答：同义改写（"介绍下你"/"你是谁干什么的"）hash 完全不同，所以用语义向量：prompt -> 512 维 embedding（本地 bigram 哈希实现，可切远程 embedding 服务），与该 model 槽位内全部缓存向量算余弦，最大且 ≥0.92 命中回放。误命中风险：问题相似但意图相反（"支持退款吗"/"不支持退款吗"）-> 阈值保守 + 命中标记 cacheHit 让看板可观测 + 生产可对高风险场景（如带否定词/敏感词）跳过缓存。多实例：本地 Caffeine 只在单机生效，生产下沉 Redis Hash 存向量（或专用向量库）+ 版本化。
- 减分信号：说不出误命中场景。

12. C3. 配额扣减为什么不用分布式锁?

- 参考回答：计数场景用 INCRBY 原子自增就够，不需要"读-判断-写"的临界区；令牌桶的读改写合并进 Lua，同样是把临界区压进单条原子操作。锁是最后手段（本项目锁的用武之地：防止同一文档并发解析——Python 侧用 asyncio.Lock，跨进程可用 SET NX PX + token 防误删）。能对比"乐观 CAS/原子命令/分布式锁"三档最好。

13. C4. Redis 挂了系统什么表现?

- 参考回答：分级降级：限流 Redis 挂 -> fail-open 放行并告警（限流是保护手段，不能因为保护系统挂了把业务全拒了，注释里写明权衡）；配额预检挂 -> 放行，事后对账兜底（软限制设计本来就允许）；app 鉴权信息挂 -> 网关回源 user 服务，DB 顶住；语义缓存挂 -> 直连上游。Redis 是增强件不是强依赖，这是架构时刻意保证的。

D. MySQL

14. D1. t_usage_record 的索引怎么设计的?

- 参考回答：唯一索引 uk_msg_id（幂等最后防线）+ 联合索引 (app_id, created_at)（按应用按时间窗口聚合/分页）+ created_at 单列（全局对账/账单聚合）。写入量大时按月分表/分区，账单聚合走预汇总表。

15. D2. 攒批写入的细节？为什么 500 条或 2 秒？

- 追问链：宕机丢数据吗 -> 批量插入冲突怎么办 -> 为什么不用 MP 自带 saveBatch
- 参考回答：内存 buffer 满 500 或定时 2s 刷一次，条数上限控内存、时间上限控延迟（对账容忍分钟级延迟，2s 是稳妥值）。宕机丢的是"未刷盘的 buffer"，但消息还在 Stream pending 里未 ACK——重启后 XCLAIM 重投，幂等键保证不重复入库（丢与重的边界要能讲清）。批量撞 uk_msg_id 冲突降级逐条、忽略重复。saveBatch 默认也是批量，但我要捕获唯一键冲突做降级，所以手写控制。

16. D3. 对账怎么做的？发现差异怎么修？

- 参考回答：每日 02:30 按应用汇总当月 DB total_tokens vs Redis 计数器；diff≠0 落 t_reconcile_diff 留痕，以 DB 为准回写 Redis（DB 有唯一索引约束、可靠性强于 Redis）。差异来源：消息未消费/未 ACK 重投后幂等跳过但计数器是 INCRBY 时机不同 / Redis 故障窗口。本质：Redis 计数是缓存态，DB 流水是事实源。

E. MQ 与异步

17. E1. 计量为什么走 MQ 而不是同步写库？

- 参考回答：推理主链路 P99 不能被写库拖累；峰值时写库削峰；billing 宕机不影响推理可用性（消息堆积恢复后补记）。代价：最终一致——所以需要幂等（防重投重复记账）、对账（兜底修正）、软限制配额（临界少量超出可接受）。

18. E2. 消息丢失的每个环节？

- 参考回答：三段论——①生产端：发布失败仅日志（设计取舍：计量允许最终一致，对账兜底；生产可本地消息表/事务消息）；② broker：Redis Stream 有 AOF 持久化级别可选 / RabbitMQ 镜像队列或仲裁队列；③ 消费端：处理成功才 XACK，处理中宕机则消息留在 pending，重启 XCLAIM 认领重投，配合幂等键不重复入库。能画出"丢/重"矩阵是加分项。

19. E3. 消息重复怎么办？

- 参考回答：重投是常态（at-least-once），所以幂等是消费端义务：Redis SETNX(msgId, TTL 48h) 快速去重 + DB uk_msg_id 唯一索引兜底（Redis 过期后重投的最后防线）。

20. E4. 为什么 usage 不怕乱序、kb 任务怕不怕？

- 参考回答：usage 是可交换的累加量，乱序无影响；kb 文档级任务天然并发安全（不同 docId），同 docId 用锁去重；真要顺序的场景（如同一会话消息流）才需要按 key 分区串行化。

F. WebFlux 与并发

21. F1. chat 服务为什么用 WebFlux 而不是 MVC?

• 参考回答：核心负载是长连接 SSE 透传——每个请求占用连接数秒到分钟级，MVC 一请求一线程会耗尽线程池（默认 200）；WebFlux 少量 EventLoop 线程支撑海量并发连接，且上游响应->客户端转发的逐 chunk 流式拷贝用 Flux 的操作符（doOnNext 统计、delayElements 回放）表达最自然。代价：全链路响应式的心智成本，阻塞调用（MyBatis）必须隔离到 boundedElastic。

• 追问：为什么不全响应式（R2DBC）? -> 渠道表读多写少、量小，blocking 隔离已够，引入 R2DBC 收益不成比例。这个取舍能讲清非常加分。

22. F2. SSE 透传时怎么做 usage 统计?

• 参考回答：不能等流结束再解析（内存里拼整包）——在 doOnNext 里对每个 delta 片段做增量累计（字符数->token 估算），流结束 doFinally 发用量事件；上游若在最后一个 chunk 带 usage 字段则优先采信。切分 token 边界导致的误差：估算按字符聚合，无边界问题。

23. F3. 背压在 SSE 场景有意义吗?

• 参考回答：有——客户端消费慢（打字机渲染卡顿）时，Reactor 的背压信号会向上游传播限流 request(n)；对 mock 回放 delayElements 就是自我限速。但对 LLM 流，上游速度通常慢于下游渲染，背压触发少；真正要防的是客户端断连（doOnCancel 取消上游订阅，止血省 token 费）。

G. 业务与计费

24. G1. 配额为什么是软限制？硬限制怎么做?

• 参考回答：预检只看已用量（Redis），请求中增量事后入账——并发下临界会少量超出（避免预扣复杂度+估算不准误杀）。硬限制方案：预扣-确认-回补（Lua 原子预扣估算值，流结束按真实值回补差额），代价是每个请求两次 Redis 往返+补偿逻辑，对演示场景收益不成比例，列为演进项。

25. G2. Token 怎么计量的？误差多大?

• 参考回答：估算 ceil(CJK×1 + 其他÷4)——中文按字≈token，英文按 4 字符≈token，BPE 分词下误差约 ±10%；上游返回真实 usage 时优先采信。诚实说明这是估算并给出校准思路（拿 tokenizer 库离线对账修正系数）比硬撑"精确"加分。

26. G3. 账单金额怎么算的？一致性怎么保证?

• 参考回答：月度定时任务聚合流水生成（prompt 0.002 元/1k、completion 0.004 元/1k 演示单价），UPSERT 幂等。一致性依赖流水表是唯一事实源——账单是派生视图，可随时重算（出账前先对账，diff 清零才出账）。

H. RAG 与 Agent（Python 侧）

27. H1. 文档切分策略？chunk 大小怎么影响效果？

• 参考回答：RecursiveCharacterTextSplitter，分隔符优先级：段落->中文句号->换行->空格；chunk 500/overlap 50。chunk 过大->检索粒度粗、噪声多、命中漂移；过小->上下文破碎、embedding 语义不完整。overlap 防止关键句被边界切断。讲得出权衡而不是背参数是加分点。

28. H2. 召回之后为什么还要过滤？怎么引用溯源？

• 参考回答：topK 召回后按相似度分数过滤（<0.15 丢），低分命中是噪声；回答 prompt 强制要求标注 [n] 引用，citations 事件回传 docName/chunkIndex/score/snippet，前端渲染成可展开引用块——可信 AI 的基本功。

29. H3. embedding 为什么默认哈希实现？生产换什么？

• 参考回答：保证离线零依赖可跑（校招项目任何人 clone 就能跑通是硬需求），bigram 哈希能捕获词序敏感的浅层语义，同义改写召回弱于真模型。生产：text2vec/bge 中文模型本地推理，或统一 embedding 服务（项目已留 RemoteEmbeddingClient 接口和 /embed 端点，切换点是配置不是代码）。

30. H4. ReAct 循环讲一下？和 Function Calling 路线比？

• 参考回答：Thought->Action->Observation 循环，AgentExecutor 驱动，工具（kb_search/calculator/time）真实执行；mock 模式下决策策略脚本化但走的是真实 AgentExecutor 链路（诚实声明）。对比：ReAct 用文本协议解析、模型无关但脆弱；原生 Function Calling 结构化、可靠但绑定协议。能说两条路线的适用面加分。

31. H5. Java 和 Python 的分工边界？跨语言任务怎么做可靠性？

• 参考回答：Java 管编排/状态/事务（文档元数据、任务派发、回调落库），Python 管模型生态（切分/embedding/向量库/Agent）。跨语言用消息队列+回调+状态机（PENDING->PARSING->READY/FAILED）解耦，失败任务可见可重试（消息不 ACK 留 pending 重投；状态字段前端可见）。这就是企业里"AI 编排服务+业务中台"的真实分工。

I. 工程与运维

32. I1. 双 profile（local/compose）的设计意图？

• 参考回答：中间件依赖做接口抽象+条件装配（@ConditionalOnProperty 切换 Redis Stream/RabbitMQ 实现），本地 MySQL+Redis 就能全链路 E2E（开发体验/CI），compose 一键起完整生产拓扑（Nacos/Nginx/RabbitMQ）。本质是六边形架构的端口适配器思想：业务逻辑不感知中间件型号。

33. I2. Nginx 配置里 SSE 相关的三件套?

- 参考回答：proxy_buffering off（响应不缓冲）、proxy_read_timeout 300s（长连接不掐）、X-Accel-Buffering: no（告知上游加速缓存别攒包）。漏一个的现象分别是"整包一次性吐出""对话到 60s 断开""偶发卡顿"——能按症状反推配置是运维加分项。

34. I3. 这个项目上线前还缺什么？（必考）

- 参考回答（按优先级）：①观测：Prometheus 指标（QPS/延迟分位/缓存命中率/消息堆积）+ 链路追踪（traceId 贯穿网关->chat->MQ->billing）；②安全：JWT 密钥 KMS 化、内网头签名/mTLS、渠道 api_key 加密存储；③可靠性：Redis/MQ 集群、服务多副本+健康探针；④成本：真实 tokenizer、渠道级成本分摊。主动暴露认知边界是最大加分项——面试官问这题就是在测你对"生产差距"的自觉。

J. 已知弱点与应答话术（【高危】必背）

弱点	一句话应答
单库多服务	演示妥协，已标注；拆库方案：事件带快照字段/服务间查询
配额软限制	预扣-回补可硬化，演示场景收益不成比例
语义缓存单机内存	接口已抽象，下沉 Redis Hash/向量库是配置级改造
token 估算	上游真实 usage 优先；离线 tokenizer 校准系数方案已想好
mock LLM/哈希 embedding	为了"clone 即跑通"；切换点都是配置（渠道表/env），架构不锁死
内网头信任	攻击路径清楚（绕网关直连），方案 mTLS/HMAC 签名，列演进项
无监控	I3 话术

L. 实战踩坑录（E2E 真实排障，面试最出彩的部分）

开发过程中真实出现、真实定位、真实修复的四个 bug——讲排障过程比讲功能更能证明工程能力：

35. L1. Redis 消费线程每 3 秒报 "command timed out"

- 现象：billing 的 XREADGROUP 消费循环持续刷超时错误，但消息最终能被消费。
- 定位：XREADGROUP ... BLOCK 3000（阻塞 3 秒等消息）与 Lettuce 客户端 timeout: 3s（命令超时 3 秒）相等——阻塞读恰好到达命令超时线，被客户端先掐死，随后重试又能读到数据，形成"每 3 秒报错但功能正常"的竞态。
- 修复：命令超时改为 10s > BLOCK 3s。
- 通用教训：所有"阻塞等待类"命令的客户端超时必须显著大于阻塞时长（生产里 Redis 慢查询同样遵循此原则）。

36. L2. @RequestParam 全部 500：'-parameters'

• 现象：接口一调就 500，日志是 "Name for argument of type [int] not specified"。

• 定位：Spring Boot 3.2 起不再自动推断参数名，要求编译期加 -parameters；改了编译配置后增量构建没重编译旧 class，现象没消失——clean 后才生效。

• 通用教训：升级框架大版本要看迁移公告；改编译参数必须 clean 验证。

37. L3. Java->Python 的 POST 请求体"消失"（422 body missing）

• 现象：kb 服务用 JDK HttpClient 调 FastAPI，检索接口稳定 422（body missing），但用 Python httpx 发同样的字节返回 200。

• 定位：先用回显服务器抓 Java 发出的原始字节（Content-Type/Content-Length/UTF-8 body 全部正确），再用中间人转发（Java->py->FastAPI）返回 200——锁定差异在 JDK 客户端与 uvicorn 之间；写最小 Java 复现矩阵（默认/强制 HTTP1.1/去掉 timeout/带 charset 四个变体），只有"默认 HTTP/2 协商"这一变体 422。

• 根因：JDK HttpClient 默认尝试 HTTP/2 升级，对 uvicorn(h11) 的 h2c 升级会让 POST body 丢失。

• 修复：客户端显式 version(HTTP_1_1)。

• 通用教训：跨语言排障三板斧——回显抓包隔离发送方、中间人二分定位链路段、最小复现矩阵锁定变量。

38. L4. 配额计数器漂移 500 = 250×2（活生生的对账案例）

• 现象：E2E 后 Redis 配额 used=500，DB 流水汇总只有 250，差一倍。

• 定位：billing 服务被强杀过，部分消息"已 INCRBY 计数、未 XACK"——重启后 XCLAIM 重投，DB 侧被唯一索引幂等吸收（流水不重复），但 Redis 计数被重复累加。

• 修复：这正是对账任务存在的意义——POST /internal/usage/reconcile 以 DB 为准覆写 Redis，diff 落 t_reconcile_diff 留痕。

• 通用教训：at-least-once + 幂等的设计下，"计数器类缓存态数据"与"事实源"必然漂移，对账是兜底而非可选。这个案例能把 E1/E2/E3 的理论题串成一个完整故事，务必讲。

K. 简历上怎么写这个项目

Prism · LLM 能力开放平台（AI 中台）| 独立开发 2026.10

技术栈：Spring Boot 3 / Spring Cloud Gateway / WebFlux / MyBatis-Plus / MySQL / Redis（Lua、Stream）/ RabbitMQ / LangChain / Chroma / Vue3 / Nginx / Docker Compose

- OpenAI 兼容网关：Spring Cloud Gateway 实现 API Key 鉴权与 Redis+Lua 令牌桶限流（按应用维度，集群安全），客户端改 base_url 即接入

- 高并发流式服务：WebFlux 全链路 SSE 透传，逐 chunk 增量统计 token；多渠道加权路由 + 连续失败摘除 + 透明故障转移

- 语义缓存：prompt 向量化 + 余弦匹配（阈值 0.92）命中即回放，重复问答零上游成本，命中率入看板

- 异步计量计费：用量走 Redis Stream/RabbitMQ（可切换），消费端 SETNX+唯一索引双重幂等、攒批入库削峰，月度配额管控 + 每日对账自动修正

- RAG 知识库：Java 编排 + Python LangChain 执行（切分->embedding->Chroma），回答带引用溯源；ReAct Agent 真实工具调用

- 工程化：双 profile（本地零中间件 E2E / compose 一键生产拓扑），Nginx SSE 反代优化，单测覆盖核心算法

（写在简历上之前，把能实测的数字补上：如语义缓存命中场景的 P95 延迟对比、攒批后的写入 TPS——数据从 E2E 压测里来，没有实测就不写数字。）

M. 本手册使用方法

• 三步自测：①遮住参考回答，逐题口述作答（电梯稿必须脱稿 30 秒讲完）；②对照参考回答与追问链，把答不上来的题标记进「已知弱点表」；③面试前一天只重看两样东西——J 节弱点话术表和 L 节实战踩坑录（排障故事是区分度最高的部分，务必能完整讲出"现象-定位-修复-教训"四段）。

• 与项目说明文档配套：架构图、模块表、实测验收记录在《Prism 项目技术说明文档》，回答"介绍一下你的项目"类问题时按"定位-架构-难点-结果"顺序引用其中数据；两份文档的数字口径一致（单测 68+28、语义缓存命中率、对账修正案例等），面试前不要凭记忆改口。

• 诚实原则：所有标记【高危】的弱点题，面试官问到就大方承认演示级取舍并给出演进方案，不要硬撑"生产级"——主动暴露认知边界是校招面试的最大加分项。

• 简历联动：K 节的简历写法可直接粘贴使用；补实测数字前先跑一遍 E2E（docs/e2e_kb_flow.py），没有实测就不要写数字。